

# Mahad AI Portfolio: Scope-Locked Implementation Blueprint

Version: 1.0

Project type: AI Product Engineering portfolio and personal AI assistant

Primary constraint: The permanent public deployment must fail safely at free-tier limits and must not create an unexpected bill.

## 1. Product definition

### 1.1 Product statement

Build a polished portfolio in which visitors can browse Mahad's work or speak to an AI assistant that answers questions about his experience, projects, product decisions, writing and technical work. The assistant accepts text or voice, supports English and Roman Urdu code-switching, grounds factual responses in a managed knowledge base, cites its sources and can synthesize responses in Mahad's cloned voice.

The product itself demonstrates:

- Product strategy and scope control
- Product design and UI/UX experience
- Full-stack engineering
- Classical ML and deep learning
- Retrieval-augmented generation
- Agentic workflows with LangGraph
- LLM observability and evaluation with LangSmith
- ML experimentation and lifecycle management with MLflow
- MLOps, CI/CD, deployment, monitoring and rollback
- Voice AI, transcription and voice cloning

### 1.2 Target users

1. Recruiters who need a quick, trustworthy view of experience and fit.
2. Engineering interviewers who want implementation depth and evidence.
3. Founders and product leaders who want to understand Mahad's product thinking.

4. Potential clients who want to evaluate design and AI delivery capability.

## 1.3 Locked MVP outcomes

The MVP is complete only when a visitor can:

1. Browse Home, Work, Case Study, Blog, About, AI Lab and Contact pages.
2. Ask a question by text or microphone.
3. Receive a grounded answer with clickable portfolio sources.
4. Inspect the retrieval evidence and LangGraph route used for that answer.
5. Hear the answer through a standard voice fallback and, when available, Mahad's cloned voice.
6. Switch among Recruiter, Engineer and Founder perspectives.
7. See a public engineering page explaining the RAG, ML and deployment lifecycle.

Mahad must be able to publish or edit a project/article in Sanity and have the relevant content automatically re-indexed without editing application code.

## 2. Scope boundaries

### 2.1 Included in version 1

- Next.js portfolio with a custom design system
- Sanity headless CMS
- Neon PostgreSQL operational database
- Qdrant vector database
- Structure-aware chunking and multilingual embeddings
- Incremental index updates and deletion handling
- FastAPI AI backend
- LangGraph orchestration
- Groq-hosted LLM and Whisper transcription
- Custom query-routing model trained from a baseline through a compact transformer
- MLflow experiment tracking and model registry in the development/CI lifecycle
- LangSmith traces and evaluation datasets with strict trace limits
- English, Roman Urdu and mixed-language test cases
- OpenVoice-based experimental voice cloning
- GitHub Actions for tests, training, ingestion and deployment

- Docker for reproducible backend and voice services
- Rate limiting, quota protection, fallbacks and privacy controls

## 2.2 Explicitly excluded from version 1

- User accounts and social login
- Payments
- A general-purpose autonomous assistant
- Sending emails, booking meetings or modifying external systems
- Fine-tuning a general-purpose LLM
- Training a speech model from scratch
- Kubernetes
- Paid AWS production hosting
- Persistent long-term voice recordings
- Unlimited conversation history
- A custom neural reranker
- A mobile application
- Multiple cooperating agents created only for demonstration

These exclusions remain locked until every MVP acceptance gate passes.

## 3. Complete technology stack

| Concern         | Technology                                    | Runs where               | Reason                                                            |
|-----------------|-----------------------------------------------|--------------------------|-------------------------------------------------------------------|
| Web application | Next.js, React, TypeScript                    | Cloudflare Pages/Workers | Strong portfolio frontend, SEO, typed UI and free static delivery |
| Styling         | Tailwind CSS, CSS variables, Radix primitives | Browser/build            | Custom, accessible design system without a heavy template look    |
| Animation       | Motion                                        | Browser                  | Controlled interaction and voice-state animation                  |

|                     |                                |                                      |                                                                                            |
|---------------------|--------------------------------|--------------------------------------|--------------------------------------------------------------------------------------------|
| CMS                 | Sanity Studio and Content Lake | Sanity Cloud                         | Structured editing, assets, APIs and webhooks                                              |
| Edge protection     | Cloudflare Worker              | Cloudflare                           | Rate limiting, CORS policy, cache headers and API proxying                                 |
| AI API              | FastAPI, Python, Pydantic      | Hugging Face Docker Space, CPU Basic | Python ecosystem for LangGraph, ML and audio; zero-cost prototype compute with cold starts |
| Agent orchestration | LangGraph                      | AI API container                     | Explicit state, branching, retries and recoverable execution                               |
| LLM                 | Groq API                       | Groq                                 | Fast free-tier inference; provider remains replaceable                                     |
| Speech recognition  | Whisper through Groq           | Groq                                 | Multilingual transcription without running a large local STT model                         |
| Relational database | Neon PostgreSQL                | Neon Cloud                           | Canonical operational data, SQL relations, migrations and scale-to-zero                    |
| Vector database     | Qdrant Cloud                   | Qdrant                               | Dedicated semantic index, metadata filters and inspectable retrieval                       |

|                    |                                             |                                         |                                                                                  |
|--------------------|---------------------------------------------|-----------------------------------------|----------------------------------------------------------------------------------|
| Embedding model    | <code>intfloat/multilingual-e5-small</code> | CI during ingestion; API during queries | Multilingual 384-dimensional embeddings with manageable CPU requirements         |
| ML baseline        | TF-IDF + logistic regression                | Training pipeline                       | Interpretable benchmark                                                          |
| ML candidate       | MiniLM/DistilBERT classifier                | Training pipeline                       | Demonstrates transformer fine-tuning                                             |
| ML serving         | ONNX Runtime                                | AI API container                        | Small, fast local inference without another network request                      |
| ML lifecycle       | MLflow OSS                                  | Local development and CI artifacts      | Experiments, metrics, artifacts and registry without a paid hosted control plane |
| Dataset versioning | Git + DVC metadata                          | GitHub repository                       | Reproducible training data without committing large derived files                |
| LLM observability  | LangSmith Developer                         | LangSmith Cloud                         | LangGraph traces, datasets, prompt tests and RAG evaluation                      |
| Voice cloning      | OpenVoice V2                                | Separate Hugging Face Docker Space      | Isolates slow speech dependencies and can fail independently                     |
| Media              | Sanity Assets                               | Sanity Cloud                            | Images and authored portfolio media stay with content                            |

|                     |                                                        |                                    |                                                                                  |
|---------------------|--------------------------------------------------------|------------------------------------|----------------------------------------------------------------------------------|
| Model artifacts     | Hugging Face model repository or public GitHub Release | Remote artifact repository         | Versioned ONNX model and label map; API downloads a pinned version at build time |
| CI/CD               | GitHub Actions                                         | GitHub-hosted runners              | Public-repository workflows, quality gates and reproducible releases             |
| Testing             | Pytest, Vitest, Playwright                             | Local and GitHub Actions           | Unit, integration and browser tests                                              |
| Python quality      | Ruff, MyPy                                             | Local and CI                       | Formatting, linting and type checks                                              |
| TypeScript quality  | ESLint, TypeScript compiler                            | Local and CI                       | Static correctness                                                               |
| API schemas         | OpenAPI generated by FastAPI                           | API and frontend client generation | Prevents frontend/backend contract drift                                         |
| Infrastructure docs | Mermaid, ADR Markdown files                            | Repository and portfolio           | Makes architecture decisions inspectable                                         |

## 4. Deployment topology

### 4.1 Public runtime

1. The browser loads the Next.js site from Cloudflare.
2. Static content is fetched from Sanity through its CDN at build time or with cached revalidation.
3. Chat and voice requests go through a Cloudflare Worker proxy.
4. The Worker applies origin checks, anonymous session limits and request-size limits.
5. The Worker calls the FastAPI AI service on a Hugging Face Docker Space.
6. FastAPI executes the ONNX routing model and LangGraph workflow.
7. LangGraph reads Neon, searches Qdrant and calls Groq only when the selected route requires it.

8. The final text response streams back to the browser using Server-Sent Events.
9. If Mahad Voice is enabled, the browser sends only the final approved text to the separate voice service.
10. The voice service synthesizes audio and streams or returns an audio file. Browser speech synthesis is the fallback.

## 4.2 Why two backend services

The agent service and voice service have different dependency, memory and latency profiles. A voice-model crash or cold start must not prevent text chat from working. Separating them also demonstrates service boundaries and independent deployment, without inventing unnecessary microservices elsewhere.

## 4.3 Free-tier behavior

- Cloudflare's free Worker allowance is capped; the application returns a friendly capacity message after internal limits.
- Hugging Face CPU Spaces sleep after inactivity, so the first request may be slow.
- Neon scales compute to zero and wakes on demand.
- Qdrant free clusters can suspend and later be deleted after prolonged inactivity; the index is always reproducible from Sanity and Neon.
- LangSmith tracing is sampled and disabled before the free trace allowance is reached.
- Groq calls are rate-limited in the Worker and API.
- No paid tier is activated for the permanent MVP.

# 5. Data ownership and database relationships

## 5.1 The three data stores

**Sanity is the authoring source of truth.** It owns human-editable projects, case studies, articles, experience entries, FAQs and architecture decisions.

**Neon PostgreSQL is the operational source of truth.** It owns ingestion state, normalized documents, canonical chunks, index versions, anonymous sessions, messages, feedback, model decisions and evaluation summaries.

**Qdrant is a derived retrieval index.** It owns embedding vectors and a compact payload used for semantic search. It can be deleted and rebuilt without losing authored content or application history.

## 5.2 Core PostgreSQL tables

### source\_documents

- `id` UUID primary key
- `sanity_id` unique text
- `source_type` enum
- `slug` text
- `title` text
- `content_hash` text
- `content_version` integer
- `published_at` timestamp
- `indexed_at` timestamp nullable
- `index_status` enum: pending, indexing, active, failed, deleted

### document\_chunks

- `id` UUID primary key
- `document_id` foreign key to `source_documents.id`
- `chunk_index` integer
- `heading_path` text array
- `content` text
- `token_count` integer
- `content_hash` text
- `embedding_model` text
- `embedding_version` text
- `qdrant_point_id` UUID unique
- `active` boolean

Unique constraint: `(document_id, chunk_index, content_hash)`.

### index\_runs

- `id` UUID primary key
- `trigger` enum: webhook, manual, scheduled, recovery
- `started_at`, `completed_at`



- documents\_added, documents\_updated, documents\_deleted
- chunks\_upserted, chunks\_deleted
- embedding\_model
- status
- error\_summary

## **chat\_sessions**

- id UUID primary key
- public\_session\_id hashed identifier
- selected\_mode
- created\_at, expires\_at
- consent\_to\_store boolean

## **messages**

- id UUID primary key
- session\_id foreign key
- role
- text\_redacted
- language
- intent
- route
- model\_version
- latency\_ms
- created\_at

## **retrieval\_events**

- id UUID primary key
- message\_id foreign key
- query\_text\_redacted
- index\_version
- top\_k
- selected\_chunk\_ids UUID array

- `scores` float array
- `retry_count`
- `answerability_decision`

## **feedback**

- `id` UUID primary key
- `message_id` foreign key
- `rating` small integer
- `reason_code`
- `comment` text nullable
- `created_at`

## **model\_releases**

- `id` UUID primary key
- `model_name`
- `version`
- `artifact_uri`
- `git_commit`
- `dataset_version`
- `metrics` JSONB
- `stage` enum: candidate, champion, archived
- `promoted_at`

## **5.3 What Qdrant stores**

Each Qdrant point contains:

- Point ID equal to `document_chunks.qdrant_point_id`
- 384-dimensional embedding vector
- `chunk_id`
- `document_id`
- `content_type`
- `project_slug`
- `persona_visibility`

- `language`
- `published_at`
- `content_version`
- A short text preview for debugging, not the canonical full text

## 5.4 Retrieval join

1. The query is embedded using the exact model/version used during ingestion.
2. Qdrant searches vectors and applies metadata filters.
3. Qdrant returns point IDs and similarity scores.
4. FastAPI maps point IDs to `document_chunks` rows in PostgreSQL.
5. PostgreSQL returns canonical chunk text and document metadata.
6. Inactive or version-mismatched chunks are discarded.
7. Selected chunks are assembled into the grounded prompt.

This division keeps semantic search fast while preserving relational integrity and auditable content in PostgreSQL.

## 6. Content and RAG lifecycle

### 6.1 Sanity schemas

Create the following schemas:

- `project`
- `caseStudy`
- `article`
- `experience`
- `education`
- `skill`
- `faq`
- `architectureDecision`
- `personalStory`
- `styleExample`
- `siteSettings`

Every indexable document includes `ragEnabled` , audience visibility, source label, canonical URL, last-reviewed date and sensitivity classification.

## 6.2 Ingestion trigger

1. Publishing in Sanity emits a signed webhook.
2. A minimal Cloudflare Worker validates the webhook and creates a GitHub `repository_dispatch` event.
3. GitHub Actions starts `ingest-content.yml` .
4. The workflow fetches the changed Sanity document and its referenced blocks.
5. A scheduled nightly reconciliation compares all published Sanity IDs against PostgreSQL so missed webhooks are repaired.

## 6.3 Normalization

- Resolve Portable Text into semantic Markdown-like text.
- Preserve heading hierarchy, lists, captions and code blocks.
- Remove navigation text and decorative labels.
- Normalize whitespace and Unicode.
- Retain English, Urdu script and Roman Urdu without translating the source.
- Attach the canonical URL and document metadata.
- Compute a SHA-256 content hash.
- Skip embedding when the hash has not changed.

## 6.4 Chunking

Use structure-aware recursive chunking:

1. Split first by document section and heading path.
2. Keep lists and their introductory sentence together.
3. Keep project facts with the relevant project identity.
4. Target approximately 350 to 500 tokens per chunk.
5. Use approximately 50 to 75 tokens of overlap only across adjacent prose sections.
6. Never mix two projects in one chunk.
7. Prefix the embedding text with document title and heading path.

Chunk settings are configuration, not hard-coded constants, and are recorded with every index run.

## 6.5 Embedding

Use the E5 input convention:

- Document: `passage: {title} | {heading_path} | {content}`
- Query: `query: {normalized_query}`

Embeddings are generated in batches during GitHub Actions. The workflow writes canonical chunks to PostgreSQL, upserts vectors to Qdrant and marks a document active only after both writes succeed.

## 6.6 Update and deletion

- Changed chunks receive new hashes and point IDs.
- New points are written before old points are deactivated.
- After validation, old PostgreSQL chunks are marked inactive and their Qdrant points are deleted.
- Deleted Sanity documents are tombstoned in PostgreSQL and removed from Qdrant.
- A failed run leaves the previous active index intact.

## 6.7 Retrieval

1. Normalize the user query.
2. Detect mode, language, intent and route with the local classifier.
3. Rewrite only ambiguous/code-switched queries when necessary.
4. Generate a query embedding locally in the API container.
5. Search Qdrant for top 12 candidates.
6. Apply audience, document-type and project filters.
7. Remove duplicates and near-identical adjacent chunks.
8. Select up to 5 chunks within the context budget.
9. Reject retrieval when scores and evidence checks are below calibrated thresholds.
10. Generate a citation for every factual claim group.

# 7. Custom ML model lifecycle

## 7.1 Model purpose

The Query Intelligence Model predicts:

- Intent

- Processing route
- Answerability likelihood
- Language category

Routes:

- `deterministic`
- `single_pass_rag`
- `agentic_rag`
- `clarify`
- `refuse`

## 7.2 Dataset

Create `data/query_router/` with:

- Human-authored recruiter and interviewer questions
- Questions about every project and article
- English, Roman Urdu, Urdu and mixed-language paraphrases
- Out-of-domain questions
- Prompt-injection examples
- Ambiguous questions that require clarification
- Synthetic augmentations clearly marked by origin

Use group-aware splits so paraphrases of one base question cannot appear across training and test sets. Keep a fully human-authored locked test set.

## 7.3 Baseline

Train TF-IDF plus logistic regression. Log to MLflow:

- Dataset version
- Feature parameters
- Class weights
- Accuracy
- Macro F1
- Per-class precision and recall
- Confusion matrix

- Calibration metrics
- Inference latency

## 7.4 Fine-tuned candidate

Fine-tune MiniLM or DistilBERT with a shared encoder and classification heads. Start with separate intent and route classifiers if multi-task training becomes unstable. Do not claim multi-task learning unless the implemented model actually shares a trained encoder across outputs.

## 7.5 Promotion gate

The transformer becomes champion only if it:

- Beats the baseline on macro F1 by a predeclared margin
- Does not regress critical refuse/out-of-domain recall
- Meets CPU latency and model-size budgets
- Passes Roman Urdu slice tests
- Remains sufficiently calibrated for confidence-based fallback

Low-confidence predictions always route to a safe default rather than being trusted blindly.

## 7.6 Packaging and serving

1. Export champion to ONNX.
2. Quantize only if accuracy remains within the accepted tolerance.
3. Publish `model.onnx`, tokenizer files, label maps, model card and metrics under an immutable version.
4. Pin that version in the backend Docker build.
5. Load one ONNX Runtime session when FastAPI starts.
6. Call it as an in-process Python service from the first LangGraph node.
7. Store model version, confidence and selected route with each request.

No HTTP model microservice is needed for the classifier at MVP scale. In-process serving is faster, cheaper and easier to operate.

## 7.7 MLflow design

Run MLflow locally during exploration and in CI for formal experiments. Store the lightweight run metadata and exported reports as build artifacts; publish approved model packages to a

versioned public model repository. The production application does not depend on an always-on MLflow server.

MLflow owns conventional ML experiments and release evidence. PostgreSQL mirrors only approved release metadata needed by the application and public engineering dashboard.

## 7.8 Monitoring and retraining

Monitor confidence, route distribution, human correction rate, out-of-domain rate and downstream retrieval success. Add corrected queries to a review queue. Retraining is manual in version 1: review labels, create a dataset version, trigger training, evaluate, approve and deploy.

# 8. LangGraph agent architecture

## 8.1 Graph state

Define a typed state containing:

- Session ID and message ID
- Raw input and optional transcript
- Sanitized input
- Selected audience mode
- Language
- Intent and classifier confidence
- Route
- Rewritten query
- Retrieval candidates
- Selected evidence
- Retry count
- Answerability state
- Draft answer
- Citations
- Final answer
- Voice status
- Error/fallback state

## 8.2 Nodes



1. `validate_input` : request size, abuse and injection checks.
2. `classify_query` : local ONNX inference.
3. `deterministic_lookup` : fetch contact/navigation/structured facts from Sanity or PostgreSQL without an LLM.
4. `normalize_query` : normalize code-switching while preserving meaning.
5. `plan_retrieval` : select metadata filters and retrieval strategy.
6. `retrieve` : Qdrant search plus PostgreSQL hydration.
7. `grade_evidence` : threshold checks followed by a constrained LLM grader only when needed.
8. `rewrite_query` : one retry maximum for weak but potentially answerable queries.
9. `generate_answer` : grounded generation using only approved evidence.
10. `verify_citations` : ensure cited chunk IDs exist and support the answer.
11. `apply_style` : apply Mahad's communication style without allowing new factual content.
12. `finalize` : redact internal details and emit response metadata.
13. `fallback` : clarify, refuse or return temporary-capacity messaging.

## 8.3 What makes it agentic

The system observes classification and retrieval results, chooses a route, selects tools/filters, evaluates evidence, conditionally retries and terminates safely. It is not described as autonomous and does not use multiple agents unless a future user problem requires them.

## 8.4 Tools available to the graph

- Portfolio semantic search
- Structured profile lookup
- Project listing/filtering
- Article listing/filtering
- Public repository metadata lookup, cached and read-only
- Architecture-document lookup

All tools are read-only in version 1.

## 9. Prompt and personality architecture

### 9.1 Separate truth from style

- RAG evidence controls facts.
- The system prompt controls role and boundaries.
- Audience mode controls depth and vocabulary.
- Curated `styleExample` records teach phrasing and code-switching.
- Voice synthesis controls sound, not content.

## 9.2 Modes

- Recruiter: concise, impact-oriented, avoids unexplained jargon.
- Engineer: architecture, code, metrics, trade-offs and limitations.
- Founder: user problem, product reasoning, MVP decisions and business value.

## 9.3 No LLM fine-tuning in MVP

Fine-tuning a general LLM would make frequently changing portfolio facts harder to update and would not replace RAG. The custom fine-tuning requirement is fulfilled by the query-routing transformer. LLM fine-tuning remains excluded until prompt/RAG evaluations demonstrate a stable style problem that examples cannot solve.

# 10. Chat experience

## 10.1 UI states

- Idle
- Listening
- Transcribing
- Classifying
- Searching portfolio
- Reviewing evidence
- Generating
- Synthesizing voice
- Speaking
- Interrupted
- Capacity reached
- Error with text fallback

## 10.2 Transport

Use REST for session creation, transcription and feedback. Use Server-Sent Events for agent progress and text tokens. SSE is sufficient because the server primarily streams in one direction. Microphone audio is uploaded as bounded chunks or one short recording in version 1; real-time bidirectional WebRTC is excluded.

## 10.3 Conversation memory

Keep the latest few turns in browser state and send a compact, bounded history with each request. Store server-side messages only after clear consent. The graph resolves references such as “What was your role there?” from the bounded conversation state. Long-term personalized memory is excluded.

## 10.4 Inspectability panel

Show a safe subset of:

- Classifier route and confidence
- Model version
- LangGraph path
- Retrieval duration
- Source titles
- Similarity scores
- Retry count
- LLM model
- Token and latency totals

Never reveal system prompts, API keys, hidden reasoning or private source content.

# 11. Voice architecture

## 11.1 Input

1. Browser requests microphone permission after a user action.
2. `MediaRecorder` captures a maximum-length recording.
3. The browser validates size and MIME type.
4. Audio goes through the protected API endpoint.
5. FastAPI calls Groq Whisper.
6. The transcript is shown for confirmation/editing before the agent runs when confidence is low.

## 11.2 Output

1. LangGraph completes and citation verification passes.
2. The browser displays text immediately.
3. Final text is split into sentence-sized segments.
4. The voice API receives text, voice-profile version and language hint.
5. OpenVoice generates audio using Mahad's consented reference profile.
6. Audio segments play sequentially.
7. Browser speech synthesis takes over if the voice service is asleep, slow or unsupported.

## 11.3 Voice data

Record 20 to 30 minutes of clean, consented audio containing English, Urdu and Roman Urdu. Keep raw recordings private. Produce a processed reference clip and derived speaker representation. The public UI explicitly labels output as a synthetic voice of Mahad.

## 11.4 Honest limitation

OpenVoice V2 does not natively promise Urdu support. Roman Urdu pronunciation and natural code-switching must be evaluated, not assumed. If quality is inadequate, cloned voice remains an experimental English mode while normal multilingual TTS handles Urdu/Roman Urdu. Voice cloning is zero-shot adaptation in MVP, not custom speech-model fine-tuning.

# 12. LLMOps with LangSmith

## 12.1 Trace hierarchy

Trace:

- Request validation
- Classifier prediction
- Query rewrite
- Retrieval
- Evidence grading
- Generation
- Citation verification
- Style application
- Voice request metadata

Redact personal data and never attach raw audio.

## 12.2 Evaluation datasets

- Recruiter questions
- Engineering questions
- Founder/product questions
- English/Roman Urdu code-switching
- Unanswerable questions
- Incorrect-premise questions
- Prompt-injection attempts
- Multi-document questions
- Citation regression cases

## 12.3 Metrics

- Retrieval hit rate at K
- Mean reciprocal rank
- Context precision and recall on labeled examples
- Answer groundedness
- Citation correctness
- Correct refusal rate
- Route agreement
- LLM calls per request
- Tokens per successful answer
- End-to-end latency
- User feedback rate

## 12.4 Trace budget

Sample production traces, trace all test-suite runs only within a controlled budget and disable remote tracing before the free monthly allowance is exhausted. The application must work when LangSmith is unavailable.

# 13. Security, privacy and reliability

- Keep all provider keys server-side.
- Use strict CORS and allowed-origin checks.

- Verify Sanity webhook signatures.
- Hash public session IDs.
- Apply IP/session rate limits at Cloudflare.
- Limit audio duration, text length and file types.
- Redact emails, phone numbers and secrets from traces where appropriate.
- Treat retrieved text as untrusted data, not instructions.
- Permit citations only from returned chunk IDs.
- Add timeouts and circuit breakers around Groq, Qdrant, Neon and voice calls.
- Provide text-only operation when voice fails.
- Provide conventional portfolio navigation when all AI services fail.
- Back up normalized content manifests and make the vector index rebuildable.
- Pin dependencies and run dependency/security scanning in CI.

## **14. Repository structure**

```
mahad-ai-portfolio/
  apps/
    web/          # Next.js portfolio and Sanity-powered pages
    studio/       # Sanity Studio
    api/          # FastAPI, LangGraph, ONNX serving
    voice/        # OpenVoice service
  packages/
    ui/           # Shared design system
    contracts/    # JSON Schema/OpenAPI-generated types
    prompts/      # Versioned prompts and mode policies
  pipelines/
    ingestion/    # Normalize, chunk, embed, upsert, reconcile
    training/     # Dataset validation, baseline and transformer
    evaluation/   # ML, retrieval and LLM evaluation
  data/
    query_router/ # Versioned metadata and permitted small
datasets
  evals/         # Human-reviewed evaluation cases
infra/
  cloudflare/
  huggingface/
  database/
docs/
  architecture/
  adr/
  runbooks/
  model-cards/
.github/workflows/
```

## 15. Step-by-step learning and implementation plan

### Phase 0: Foundation and architecture literacy

Learn:

- HTTP, REST, SSE and CORS
- Containers and environment variables

- Relational versus vector databases
- Embeddings, cosine similarity and RAG failure modes
- Training versus inference versus fine-tuning

Build:

- Public monorepo
- Issue board and milestones
- Architecture diagram
- ADRs for each external service
- `.env.example` with no secrets
- Local Docker Compose for PostgreSQL-compatible testing and Qdrant

Gate:

- Every component has one written responsibility.
- Local services start from documented commands.
- No application feature is implemented yet.

## Phase 1: Conventional portfolio and design system

Learn:

- Next.js routing and rendering
- TypeScript domain types
- Responsive layout, accessibility and performance
- Design tokens and component APIs

Build:

- Home, Work, Case Study, About and Contact
- Typography, color, spacing and motion tokens
- Accessible navigation and forms
- Static placeholder content
- Playwright smoke test and Lighthouse CI

Gate:

- The portfolio is useful without AI.
- Mobile, keyboard navigation and core performance checks pass.



## Phase 2: Sanity CMS and blog

Learn:

- Headless CMS concepts
- Content schemas and references
- GROQ queries
- Preview and publishing workflows

Build:

- Locked schemas from section 6
- Sanity Studio
- Blog index/detail pages
- Project and case-study rendering
- Preview mode and image handling

Gate:

- Publishing a project or article updates the site without a code change.
- Invalid or incomplete content is blocked by schema validation.

## Phase 3: PostgreSQL operational layer

Learn:

- Relational modelling, foreign keys, indexes and migrations
- Transactions and idempotency
- Async database access

Build:

- Neon project and development branch
- SQLAlchemy 2.x models and Alembic migrations
- Tables from section 5
- Repository layer and health check
- Seed and reset scripts for local development only

Gate:

- Migrations create a clean database.

- Foreign-key and uniqueness tests pass.
- No vector search exists yet.

## Phase 4: Offline RAG ingestion prototype

Learn:

- Cleaning, chunking, tokenization and embeddings
- Embedding-model input conventions
- Idempotent ETL design

Build:

- Fetch one Sanity document
- Normalize Portable Text
- Chunk by heading hierarchy
- Generate E5 embeddings locally
- Save document/chunk records to PostgreSQL
- Unit tests for headings, lists, overlap and hashes

Gate:

- Running ingestion twice creates no duplicates.
- Changing one section reprocesses only affected content.

## Phase 5: Qdrant and end-to-end retrieval

Learn:

- Vector collections, dimensions, distance metrics and payload indexes
- Approximate nearest-neighbor retrieval
- Metadata filtering and score calibration

Build:

- Qdrant collection with cosine distance and 384 dimensions
- Vector upsert/deletion
- PostgreSQL hydration by Qdrant point ID
- Command-line retrieval inspector
- A labeled retrieval test set

Gate:

- Top-K retrieval returns correct project evidence for the agreed test set.
- Deleting Qdrant and running rebuild restores the index.
- No LLM is used yet.

## Phase 6: Automated content lifecycle

Learn:

- Webhooks, CI triggers, reconciliation and rollback
- Secrets in CI

Build:

- Signed Sanity webhook
- Cloudflare trigger endpoint
- GitHub ingestion workflow
- Nightly reconciliation
- `index_runs` logging
- Failed-run and stale-chunk recovery

Gate:

- Create, update and delete tests all synchronize Sanity, PostgreSQL and Qdrant.
- A failed update leaves the last good index queryable.

## Phase 7: Query-router dataset and ML baseline

Learn:

- Label design, leakage, imbalanced metrics and calibration
- Scikit-learn pipelines
- MLflow experiment tracking

Build:

- Label guide
- Human-authored dataset and locked test set
- TF-IDF/logistic-regression baseline
- MLflow runs and model card

- Error-analysis notebook

Gate:

- Dataset has passed duplicate/leakage checks.
- Baseline metrics and known weaknesses are documented.

## **Phase 8: Transformer fine-tuning and MLOps**

Learn:

- Tokenizers, transformer fine-tuning and class weighting
- ONNX export and quantization
- Model registry, champion/challenger and rollback

Build:

- MiniLM/DistilBERT candidate
- Hyperparameter experiments
- Slice evaluation for Roman Urdu and refusals
- Promotion script
- ONNX artifact and immutable release
- CI inference-contract test

Gate:

- Candidate either earns promotion by the declared rules or the baseline remains champion.
- The FastAPI service can load the champion and return route/confidence locally.

## **Phase 9: FastAPI and LangGraph text assistant**

Learn:

- FastAPI dependency injection and streaming
- State machines, conditional edges and retries
- Grounded prompting and tool boundaries

Build:

- Typed graph state and nodes from section 8
- Deterministic route

- Single-pass RAG route
- Agentic route with one rewrite retry
- Citation verifier
- SSE endpoint
- Provider timeouts and fallbacks

Gate:

- The assistant answers supported questions with citations.
- Unsupported questions clarify or refuse.
- The graph cannot loop indefinitely.
- It works without LangSmith enabled.

## **Phase 10: LangSmith and LLM evaluation**

Learn:

- Trace structure, datasets, evaluators and regression testing
- Difference between application monitoring and LLM evaluation

Build:

- Redacted trace hierarchy
- Curated evaluation datasets
- Retrieval, groundedness, citation and refusal evaluations
- Prompt/version comparison
- Monthly trace-budget circuit breaker

Gate:

- A prompt or retrieval regression blocks release.
- Trace data contains no raw audio or secrets.

## **Phase 11: Production chat UI**

Learn:

- Streaming UI, optimistic state and accessible async feedback
- API contract generation and error recovery

Build:

- Mode selector
- Suggested questions
- Streaming transcript
- Source cards
- Inspectability panel
- Feedback controls
- Capacity/cold-start/error states

Gate:

- Chat works on desktop and mobile.
- Every failure has a comprehensible recovery path.

## **Phase 12: Voice input**

Learn:

- Browser media APIs, codecs, permissions and transcription UX
- Audio privacy and bounded uploads

Build:

- Push-to-talk recording
- Duration and size limits
- Whisper transcription endpoint
- Editable transcript
- English/Roman Urdu test set

Gate:

- Voice questions reliably become editable text.
- Denied microphone permission never breaks chat.

## **Phase 13: Voice cloning and output**

Learn:

- TTS, speaker representations, zero-shot cloning and streaming audio
- Objective versus human voice evaluation

Build:

- Private recording protocol
- OpenVoice Docker service
- Versioned Mahad voice profile
- Sentence-level synthesis queue
- Playback, stop and replay controls
- Browser TTS fallback
- Synthetic-voice disclosure

Gate:

- English quality passes human review.
- Roman Urdu limitations are measured and disclosed.
- Voice-service failure does not affect text answers.

## **Phase 14: Deployment and operations**

Learn:

- Multi-service deployment, secrets, health checks, logs and runbooks
- Rate limits, circuit breakers and recovery

Build:

- Cloudflare deployment
- Agent API Docker Space
- Voice Docker Space
- Neon and Qdrant production configuration
- Deployment workflows
- Health dashboard
- Index rebuild, model rollback and provider-outage runbooks

Gate:

- Fresh deployment from documented steps succeeds.
- No secrets exist in repository history.
- Free-tier exhaustion causes service degradation, not charges.

## **Phase 15: Portfolio case study and launch**

Learn:

- Communicating engineering evidence and limitations
- Product analytics and iteration

Build:

- Public architecture page
- MLflow experiment summary
- LangSmith evaluation summary
- RAG lifecycle explorer
- Cost/latency comparison of routed versus all-LLM traffic
- Demo video and technical README

Gate:

- Claims are backed by metrics or explicitly labeled hypotheses.
- A recruiter understands the value in under one minute.
- An engineer can inspect the implementation and reproduce core tests.

## **16. Test strategy**

### **Unit tests**

- Sanity normalization
- Chunk boundaries and hashes
- Classifier preprocessing and labels
- Route decisions
- Citation validation
- Prompt-context construction

### **Integration tests**

- PostgreSQL migrations
- Qdrant upsert/search/delete
- Sanity-to-index synchronization
- Groq provider adapter with recorded mocks
- LangGraph route completion
- Voice-service contract

### **End-to-end tests**



- Browse project and article
- Ask deterministic question
- Ask grounded RAG question
- Ask multi-source question
- Ask unanswerable question
- Submit voice question
- Recover from sleeping voice backend

## Evaluation tests

- Router macro F1 and slice metrics
- Retrieval hit rate and MRR
- Groundedness and citation correctness
- Refusal correctness
- English/Roman Urdu behavior
- LLM-call avoidance and latency

## 17. Release sequence

- Release 0.1: Conventional portfolio
- Release 0.2: Sanity CMS and blog
- Release 0.3: Reproducible RAG ingestion and retrieval inspector
- Release 0.4: Custom ML router
- Release 0.5: LangGraph text assistant
- Release 0.6: LangSmith evaluations and public inspectability
- Release 0.7: Voice input
- Release 0.8: Experimental Mahad voice
- Release 1.0: Hardened deployment and complete case study

Each release must remain deployable and presentable. Do not wait until voice cloning is complete before sharing the project.

## 18. Cost and platform caveats

The selected services have current free options, but free allowances and terms can change. Before creating each production resource, re-check the official pricing page and do not

attach a payment method unless a provider requires it and supplies a true hard spending cap.

Current planning references:

- Cloudflare Workers Free: 100,000 daily requests and capped free behavior: <https://developers.cloudflare.com/workers/platform/pricing/>
- Cloudflare Pages static assets and Functions: <https://developers.cloudflare.com/pages/functions/pricing/>
- Neon Free: no time limit or credit card, with scale-to-zero: <https://neon.com/pricing> and <https://neon.com/docs/introduction/scale-to-zero>
- Qdrant free cluster: no credit card; free-cluster lifecycle caveats: <https://qdrant.tech/documentation/cloud/create-cluster/>
- Sanity free-plan limits and blocking behavior: <https://www.sanity.io/docs/content-lake/technical-limits> and <https://www.sanity.io/docs/platform-management/plans-and-payments>
- Hugging Face Spaces hardware and sleep behavior: <https://huggingface.co/docs/hub/spaces-overview>
- LangSmith Developer trace allowance: <https://www.langchain.com/pricing>
- GitHub Actions public-repository usage: <https://docs.github.com/en/actions/concepts/billing-and-usage>
- OpenVoice V2 license and language claims: <https://github.com/myshell-ai/OpenVoice>

## 19. Final architecture decisions

1. Sanity owns authored content; PostgreSQL owns operational truth; Qdrant is rebuildable.
2. The custom trained model performs economically meaningful routing.
3. The classifier runs in-process as ONNX; it is not an unnecessary microservice.
4. LangGraph is used for stateful branching and recovery, not decorative multi-agent complexity.
5. MLflow owns ML experiments; LangSmith owns LLM/agent traces and evaluations.
6. Voice is a separate failure domain with text and browser-TTS fallbacks.
7. Custom LLM fine-tuning, Kubernetes and paid AWS production are outside MVP.
8. Every phase ends in a working, demonstrable release.
9. Production features must work when observability vendors are unavailable.

10. Any proposed addition must replace an existing scope item or wait until version 1 is complete.